

实验代码思路:

1、题目要求: 返回只有最高位置是 1 其余位置是 0 的掩码 0x80000000。

实现思路:

1 的二进制只有末位是 1, 只需要把 1 向左位移 31 位即可得到目标。

2、题目要求: 用~和&实现按位异或, 即用~和&表达 $x \oplus y$ 。

实现思路:

异或的意思是相同为 0, 不同为 1, 也就是 x 和 y 对应的每一位, 必须至少有一个是 0 且至少有一个是 1 时, 才为 1。($\sim x \& \sim y$) 在某一位上为 1, 表示 x 、 y 在该位都是 0, 取反 $\sim(\sim x \& \sim y)$ 就表示至少有一个是 1; ($x \& y$) 在某一位上为 1, 表示 x 、 y 在该位都是 1, 取反 $\sim(x \& y)$ 就表示至少有一个是 0。用相与符号结合, $\sim(\sim x \& \sim y) \& \sim(x \& y)$, 两个条件都满足时为 1, 即可用来表示异或。

3、题目要求: $x < 0$ 时返回 $-x$, 否则返回 0。

实现思路:

当 $x < 0$ 时, 第 31 位为 1, $x \geq 0$ 时, 第 31 位为 0, 即 $x \gg 31$ 后, 在 x 为负时得到全 1, 非负时得到全 0, 可以当掩码, $-x$ 可以用 $\sim x + 1$ 来表示。把掩码和 $\sim x + 1$ 相与, 负数时掩码全 1, & 保留 $(\sim x + 1)$, 最终返回 $-x$, 非负时掩码全 0, & 得到 0, 最终返回 0。

4、题目要求: 一个 32 位整数有 4 个字节, 编号 0 (最低位字节) 到 3 (最高位字节)。把第 src 个字节的值复制到第 dst 个字节, 其他字节保持不变。

实现思路:

一个字节有 8 位, 第 k 个字节占据第 $k*8$ 位到第 $k*8+7$ 位, 左移一位等于乘 2, 所以 $k*8$ 等价于 $k \ll 3$ 。首先取出 src 字节, 用 $x \gg (src \ll 3)$ 把第 src 个字节挪到最低位, 再用 $\& 0xFF$ 只保留最低 8 位, 把多余的高位清掉, 得到只含源字节的 a 。然后清空 dst 字节, 通过在 $0xFF \ll (dst \ll 3)$ 再取反, 构造一个在 dst 字节位置全 0、其它位全 1 的掩码, 用其和 x 相与, 把 x 的目标字节清零, 保留其它位, 得到 b 。最后把取出的 a 移到 dst 位置, $a \ll (dst \ll 3)$, 这样对应 dst 个字节位置上就是第 src 个字节的值, 和 b 相或即可。

5、题目要求: 实现逻辑右移——把 x 右移 n 位, 左边空出的位补 0, 不论正负。

实现思路:

先让 $x \gg n$ 。如果 x 非负, 算术右移补的也是 0, 结果已经正确; 如果 x 是负数, 算术右移会让高位补进 n 个 1, 需要制造一个高 n 位为 0、低 $32-n$ 位为 1 的掩码, 并与结果相与来清除高位的 n 个 1。先用一个全是 1 的掩码 ~ 0 左移 $(32-n)$ 位 $\sim 0 \ll (32-n)$, 得到一个高 n 位为 1、低 $32-n$ 位为 0 的掩码, 再取反即可。但是当 $n=0$ 时 $(32-n)$ 等于 32, 左移 32 位是未定义行为, 为了避免, 我把 $32-n$ 写成 $31-n+1$, 分两次左移, 效果等价于移 $32-n$ 位, 但移位置量不会超过 31, 用 $\sim n + 1$ 表示 $-n$ 。

6、题目要求: 把每个字节里的高 4 位和低 4 位进行互换。

实现思路:

因为是对每个字节独立操作，需要构造一个掩码，让每字节低 4 位为 1、高 4 位为 0，即 0x0F0F0F0F。因为常量不能超过 255，所以用小常量 0x0F 拼：把它分别左移 8、16、24 位再相加，就得到 0x0F0F0F0F。接着用 $x \gg 4$ 让每个字节的高 4 位被右移到低 4 位的位置，并用 $\& a$ 清空高 4 位的值，得到在每个字节的低 4 位的位置有高 4 位值，高 4 位为 0 的结果。然后将原本的数和 a 相与去除高 4 位后，把每个字节的低 4 位左移到高 4 位，这样就得到在每个字节的高 4 位的位置有低 4 位值，低 4 位为 0 的结果。两部分取或就是最终 4 个字节同时交换的结果。

7、题目要求：从最低位往上看，找到第二个值为 0 的位，返回一个只有这一位为 1 的掩码；如果 0 的个数不足两个，返回 0。

实现思路：

x 的 0 位就是 $\sim x$ 里的 1 位，可以把这道题看成找 $\sim x$ 里从低位到高位第二个 1。先用 $\sim x \& (\sim x + \sim 0)$ 去除最低位的 1，因为一个数减 1 时，最低位那个 1 被借走变成 0，它右边的 0 全部变成 1，左边不变，所以和原数相与后，最低位 1 变成 0，其他位不受影响。然后再用 $y \& (\sim y + 1)$ 取出最低位的 1，取反加 1 会让最低位 1 及其右边保持不变、左边翻转，所以相与后只有最低位 1 被保留，其他位都是 0，得到的那个 1 位位置就是 x 的次低 0 位。

8、题目要求：统计 x 的二进制中 1 的个数，若为偶数返回 1，若为奇数返回 0。

实现思路：

异或的规则表是 $0 \wedge 0 = 0, 0 \wedge 1 = 1, 1 \wedge 0 = 1, 1 \wedge 1 = 0$ ，而两个数相加除以 2 的余数的规则表也是 $0 + 0 = 0, 0 + 1 = 1, 1 + 0 = 1, 1 + 1 = 0$ ，完全一样。所以异或可以当作只看奇偶的加法，把 x 的各位数两两异或合并，奇偶性不变，只要把 32 位全部异或到一起，就可以得到 1 的个数的奇偶性，结果为 0 说明有偶数个 1，为 1 说明有奇数个 1。通过对折来实现 32 位的异或合并，把高 16 位和低 16 位异或，有效位数从 32 降到 16；再对折成 8、4、2、1 位，最后最低位就是所有位的异或，也就等于 1 的个数的奇偶性，最低位是 1 表示奇数个 1，0 表示偶数个 1。最后 $x \& 1$ 就是 1 的个数的奇偶性（0=偶，1=奇），但题目要求偶=1、奇=0，所以再 $\wedge 1$ 翻转实现目标。

9、题目要求：把 x 循环右移 n 位，右边移出去的位从左边补回来。

实现思路：

循环右移可以看成两步，第一步是普通的将 x 右移，第二步是把移出去的低 n 位左移搬到高 n 位。右移部分 $x \gg n$ 需要用第 5 题做过的逻辑右移令高 n 位为 0，左移部分 $x \ll (32 - n)$ 把原来的低 n 位搬到高 n 位，低 $(32 - n)$ 位为 0，为了防止 $n = 0$ 溢位移量变成 32，先左移 $31 + \sim n + 1$ ，再左移 1 位，将两部分相或得到目标值。

10、题目要求：把非负 x 四舍五入到 2^n 的倍数；如果正好在两个倍数中间，选商为偶数的那个

实现思路：

四舍五入的通用做法是加偏移再截断是，先加半个单位，再向下取整，要把 x 四舍五入到 2^n 的倍数，单位是 2^n ，半个单位就是 $2^{(n-1)}$ ，可以用 $a = 1 \ll (n + \sim 0)$ 表示。但是题目要求正好在中间时需要看商的奇偶性决定是否进位，如果直接加 a ，在中间时就会直接进位，所以需要额外减 1，当商是偶数是不变，商是奇数时加 1，从而得到偶数商。用 $b = (x \gg n) \& 1$ 取出商的奇偶性，再算 $m = x + a + b + \sim 0$ 得到偏移后的值，最后用 $(m \gg n) \ll n$ 截断，右移 n 位，相当于，除以 2^n 向下取整，再左移 n 位，乘回 2^n ，就得到了 2^n 的倍数。

11、题目要求：精确计算数学上的中点 $(x+y)/2$ ，不能溢出。特别地，如果中点正好在两个整数中间（即 x 、 y 一奇一偶），要选离 x 较近的那个整数。

实现思路：

$x+y$ 可能会溢出，所以通过 $(x>>1)+(y>>1)$ 让 x 、 y 各自整除 2，如果 x 和 y 都是偶数那结果刚好就是 $(x+y)>>1$ ，如果 x 和 y 都是奇数，那么 $x>>1$ 和 $y>>1$ 各自少了 0.5，需要最终结果+1，用 $x\&y\&1$ 补回来。如果 x 和 y 一奇一偶，真实中点应该是 $(x>>1)+(y>>1)+0.5$ ，要选取距离 x 较近的整数，也就是如果 $x>y$ 向上取整，结果是 $(x>>1)+(y>>1)+1$ ，如果 $x<y$ 向下取整，结果是 $(x>>1)+(y>>1)$ 。

因此需要判断 x 和 y 的奇偶性，并比较 x 与 y 的大小，还要防止溢出。

先用 $q=(x\^y)\&1$ 来判断 x 和 y 的奇偶性， $q=0$ 时说明最低位相同， x 和 y 同奇同偶， $q=1$ 时说明最低位不同， x 和 y 一奇一偶。然后讨论一奇一偶的情况，构造一个 fin 让 x 大于 y 时为 1。 x 和 y 异号时， $sign=(x>>31)\^ (y>>31)=1$ ， $x\geq 0, y<0$ ，一定 $x>y$ ， $fin=1$ ； $x<0, y\geq 0$ ，一定 $x<y \rightarrow fin=0$ ，所以 fin 等同于 x 的符号位取反。 x 和 y 同号时， $sign=0$ ， $x-y$ 不会溢出，直接用右移位取出 $(x-y)$ 的符号位， $m=((x+\sim y+1)>>31)\&1$ ，其中 $x+\sim y+1=x-y$ ，符号位为 1 表示 $x<y$ ，所以 $fin=\sim m$ 表示 $x\geq y$ ，所以 $fin=(sign\&\sim x1)|(\sim sign\&\sim m)$ ，两种情况下 $x>y$ 时 fin 为 1。最后返回 $half+(q\&fin)$ ， $q=0$ 时 x 和 y 同奇同偶，跳过修正， $q=1$ 时 x 和 y 一奇一偶，按 fin 决定是否+1。

12、题目要求：判断 x 是否在 a 、 b 构成的闭区间内， a 、 b 顺序任意。

实现思路：

这题首先需要在保证不溢出的情况下比较 a 、 b 大小，确定 min 和 max ，然后再判断 $min\leq x\leq max$ 。

首先确定 a 和 b 的大小。用右移提取出 a 和 b 的符号位，当 a 和 b 异号时， $a1\^b1=1$ ，如果 $a1=1$ 说明 $a<0<b$ ，如果 $a1=0$ 说明 $a>0>b$ 。当 a 和 b 同号时，不用担心溢出， $a1\^b1=0$ ，可以直接右移提取 $(a+\sim b)$ 的符号位，符号位如果为 1 说明 $a\leq b$ ，合并可得 $m=((a1\^b1)\&a1)|(\sim(a1\^b1)\&((a+\sim b)>>31))$ ， $m=1$ 表示 $a\leq b$ ， $m=0$ 表示 $a>b$ 。

接下来根据 m 从 a 和 b 里挑出小的当 min 、大的当 max 。先算 $d=a\^b$ ，它记录了 a 和 b 哪些位不一样，再算 $d\&\sim m$ ：当 $m=1$ 时， $\sim m$ 是 0， $d\&\sim m$ 就是 0；当 $m=0$ 时， $\sim m$ 全是 1， $d\&\sim m$ 就是 d 本身。把这个结果拿去和 a 异或， $m=1$ 时 $a\^0$ 还是 a ，等于没变； $m=0$ 时 $a\^d=a\^(a\^b)=b$ ，等于把 a 换成了 b 。所以 min 写成 $a\^(d\&\sim m)$ ，就实现了 $m=1$ 时取 a 、 $m=0$ 时取 b 。同理 max 写成 $b\^(d\&\sim m)$ ， $m=1$ 时取 b 、 $m=0$ 时取 a 。

然后分别判断 $min\leq x$ 和 $x\leq max$ 。判断 $min\leq x$ 时，先取 min 和 x 的符号位，异号时由符号位直接决定，同号时看 $(min+\sim x)>>31$ 的符号位，为 1 说明 $min<x$ ，合并得到 $r1$ ， $r1=1$ 表示 $min\leq x$ 。判断 $x\leq max$ 时，先取 x 和 max 的符号位，异号时看符号位，同号时看 $(x+\sim max)>>31$ 的符号位，为 1 说明 $x<max$ ，合并得到 $r2$ ， $r2=1$ 表示 $x\leq max$ 。闭区间要求两个条件同时成立，所以用 $r1\&r2$ 表示同时满足，因为 $r1$ 和 $r2$ 可能是非 0 的掩码值，所以要用 $!!$ 归一化成 0 或 1 返回。

13、题目要求：返回 $x*5$ ；溢出时正溢出返回 INT_MAX ，负溢出返回 INT_MIN 。

实现思路：

先用 $x*4+x$ 来表示 $x*5$ ，即 $(x<<2)+x$ ，然后判断是否溢出，若溢出用饱和值替换掉结果。

判断溢出需要一个阈值，正溢出发生在 $x*5$ 超过 INT_MAX ，也就是 x 超过 $INT_MAX/5$ 。 $INT_MAX/5$ 向下取整是 429496729，即 $0x19999999$ ，需要用 $0x66$ 拼接得到，通过 $0x66<<8|0x66$ 得到 $0x6666$ ，再通过 $0x6666<<16|0x6666$ 得到 $0x66666666$ ，最后右移 2 位得

到 0x19999999，正好是阈值。

正溢出等价于 $x \geq t+1$ ，其中 $t = \text{INT_MAX}/5$ 。当 $x \geq t+1$ 时， $x-t-1 \geq 0$ ，符号位是 0；当 $x < t+1$ 时，符号位是 1。对符号位取反，让正溢出时标志变成 1。同时，正溢出只可能发生在 x 为正的时候， x 为负时不可能正溢出。为了排除 x 为负的干扰，把取反后的结果再和 x 的符号掩码取反相与： x 为正时取反掩码全 1，保留标志； x 为负时取反掩码全 0，把标志清掉。这样得到的标志位就只在 x 为正且 $x \geq t+1$ 时为 1，准确表示正溢出。

负溢出等价于 $x \leq -(t+1)$ 。当 $x+t \leq -1 < 0$ 时，符号位是 1，否则符号位是 0。所以符号位为 1 就代表负溢出。同样地，负溢出只可能发生在 x 为负的时候， x 为正时不可能负溢出。为了排除 x 为正的干扰，把符号位和 x 的符号掩码相与： x 为负时掩码全 1，保留标志； x 为正时掩码全 0，把标志清掉。这样标志位就只在 " x 为负且 $x \leq -(t+1)$ " 时为 1，准确表示负溢出。两个标志合并得到溢出掩码 fin ，没溢出时是全 0，溢出时是全 1。

最后决定输出， $x*5$ 溢出时会得到一个错误的值。饱和值 s 根据 x 的符号取 INT_MAX 或 INT_MIN ：用 $n \wedge (\sim(1 << 31))$ 得到，其中 n 是 x 的符号掩码， $\sim(1 << 31)$ 是 INT_MAX 。 x 为正时 $n=0$ ， $0 \wedge \text{INT_MAX} = \text{INT_MAX}$ ； x 为负时 n 是全 1， $1 \wedge \text{INT_MAX} = \text{INT_MIN}$ 。然后用 $y \wedge ((y \wedge s) \& \text{fin})$ 来输出， fin 为 0 没溢出时， $(y \wedge s) \& 0 = 0$ ，返回 $y \wedge 0 = y$ ； fin 为全 1 溢出时， $(y \wedge s) \& 1 = y \wedge s$ ，返回 $y \wedge (y \wedge s) = s$ ，从而实现没溢出返回 y 、溢出返回饱和值 s 。

14、题目要求：判断 $x+y+z$ 的精确值是否超过 INT_MAX （返回 1）、低于 INT_MIN （返回 -1），否则返回 0。

实现思路：

分两步做加法时，中间步骤可能溢出，但最终结果可能因为两次溢出方向相反而回到范围内，所以需要判断有没有溢出，以及溢出的方向有没有被抵消。

对第一步加法 $x+y$ ，溢出条件是 x 、 y 同号但结果 $r1$ 与它们异号。用位运算表达： $\sim(x \wedge y)$ 的符号位表示 x 、 y 同号， $x \wedge r1$ 的符号位表示 $r1$ 和 x 异号，两者相与后右移 31 位得到掩码 $t1$ ，全 1 表示溢出。再用 x 的符号 $x1$ ，判断 x 的正负性，区分方向， $p1 = t1 \& \sim x1$ 是正溢出， $n1 = t1 \& x1$ 是负溢出。

接着算第二次加法 $r2 = r1 + z$ ，用同样的方法判断溢出。 $\sim(r1 \wedge z)$ 表示 $r1$ 、 z 同号， $r1 \wedge r2$ 表示 $r2$ 和 $r1$ 异号，相与右移 31 位得到 $t2$ 。用 z 的符号 $z1$ ，判断 z 的正负性，区分方向， $p2$ 是正溢出， $n2$ 是负溢出。四个标志 $p1$ 、 $n1$ 、 $p2$ 、 $n2$ 都是 0 或全 1 掩码。

最后判断真实溢出方向。两次方向相反的溢出会互相抵消：比如第一次正溢出、第二次负溢出，真实值可能回到范围内。所以最终正溢出是第一次正溢出且第二次没有负溢出 ($p1 \& \sim n2$)，或第一次没溢出而第二次正溢出 ($p2 \& \sim n1$)，合起来 $\text{final_p} = (p1 \& \sim n2) | (p2 \& \sim n1)$ 。最终负溢出是第一次负溢出且第二次没有正溢出 ($n1 \& \sim p2$)，或第一次没溢出而第二次负溢出 ($n2 \& \sim p1$)，合起来 $\text{final_n} = (n1 \& \sim p2) | (n2 \& \sim p1)$ 。

返回值时，用 $q | \text{final_n}$ 统一处理。 $q = !\text{final_p}$ 可以把正溢出掩码归一化成 0/1。正溢出时 $q=1$ ， $\text{final_n}=0$ ，返回 1；负溢出时， $\text{final_n}=-1$ ， $q=0$ ，返回 -1；都没溢出时 $q=0$ 、 $\text{final_n}=0$ ，返回 0，满足题目要求。

15、题目要求：返回浮点数 $f^{3/2}$ 的位表示。用就近舍入，保留 ± 0 的符号，NaN 原样返回。

实现思路：

先把浮点数拆出符号、阶码、尾数三段，分开处理，然后分情况讨论。NaN/Inf（阶码全 1）直接返回。

对非规格化数（阶码为 0），没有隐藏位，直接把尾数乘 3 得到 A，再右移 1 位得到整数部分 n，用就近舍入处理最低位：当 A 的最低位和 n 的最低位都是 1 时进位。如果进位后 n 的位数超过 23 位，说明结果变成规格化数，阶码从 0 变成 1，否则结果还是非规格化。

对规格化数，先补上隐藏位得到 24 位的尾数 m，乘 3 得到 A。乘 3 后尾数可能超出一位，所以分两种情况：如果 A 没有超过 2^{24} ，乘 3 后没有进位到更高阶，直接右移 1 位再按就近舍入；如果 A 超过 2^{24} ，说明乘 3 后进位了，阶码要加 1，尾数右移 2 位再舍入，被舍掉的余数大于一半就进位，等于一半看最低位是否为奇数。最后拼装符号、阶码、尾数，如果阶码溢出到 255 (Inf) 就直接返回 Inf。

16、题目要求：把浮点数舍入到最近整数，正中间舍入到偶数，返回浮点表示；NaN/Inf 原样返回，舍入到 0 时保留符号。

实现思路：

按阶码 e 分类讨论，先取出符号位、阶码、尾数。

e=255 是 NaN/Inf，原样返回。

e>=150 时，阶码减去偏置 127 大于等于 23，说明浮点值已经大于等于 2^{23} ，小数部分已经不存在，本身就是整数，也原样返回。

e<=125 时，指数小于 -2，浮点值绝对值小于 0.5，舍入到 0，但要保留符号位，直接返回 sign。

e=126 时值在 0.5 到 1 之间，要单独处理。尾数为 0 说明正好是 0.5，按就近偶数舍入到 0；尾数非 0 说明大于 0.5，舍入到 1，再加上符号位。

e 在 127 到 149 之间时，浮点值在 1 和 2^{23} 之间，需要进行舍入。先给尾数补上隐藏位得到 24 位的 m。右移 shift 位 (shift=150-e) 得到整数部分 n，同时用掩码取出被舍掉的低 shift 位作为余数 rem。半值 half 是 $1 \ll (\text{shift}-1)$ 。按就近舍入规则判断是否进位：rem 大于 half 或者 rem 等于 half 且 n 是奇数则进位。最后重新组装浮点表示。此时 n 是整数部分的尾数，但可能因为进位位数增加，阶码要相应变化。k=e-127 是实际的指数值，把 n 的高位部分作为新的阶码增量加到原阶码 e 上，低位部分作为新的尾数，将 $(e+(n \gg (k+1))) \ll 23$ 加上 $(n \& ((1 \ll k)-1)) \ll (23-k)$ ，再与符号位相或。

17、题目要求：把 int 转成 float 的位表示。

实现思路：

0 直接返回 0。先取符号位，并对负数取绝对值，然后找最高位 1 的位置来确定阶码。float 的阶码是偏置表示，偏置是 127。用循环左移的办法：每次把 ux 左移一位，直到最高位变成 1，因为 int 有 31 位有效数字，加上偏置后是 158，左移 i 位后阶码就是 158-i，将阶码 e 从 158 开始每次减 1，循环结束时可以得到正确的偏置阶码 e。

接下来处理尾数的舍入。float 的尾数只有 23 位，而 int 的有效数字最多 31 位，所以要丢掉低 8 位。先取低 8 位作为余数 rem，判断是否要进位，rem 大于 0x80 说明超过半，进位，rem 等于 0x80 说明正好是半，看保留部分的最低位（即 ux 的第 8 位）是否为 1，是 1 就进位，在保留部分加 1，凑成偶数。

进位后可能出现最高位又向上溢出的情况：比如原来最高位在第 23 位，进位后到了第 24 位，这时阶码要加 1。用 ux<old 判断是否溢出，因为加到最高位后 ux 会变小（符号位翻转），这个技巧利用了补码加法的特性。

将符号左移 31 位、阶码左移 23 位、尾数取低 23 位 $((ux \gg 8) \& 0x7FFFFFFF)$ ，相或得到结果。

18、题目要求：统计 32 位中 1 的个数。例：bitCount(5) = 2

实现思路：

采用分治的办法，通过构造掩码，把 32 位先分成 16 个 2 位小组，每组分别数 1 的个数；然后不断合并相邻小组，依次变成 4 位一组、8 位一组、16 位一组，最后合成总数。

构造掩码需要用小常量拼接： $m1 = 0x55((0x55 \ll 8))$ 先得到 0x5555，再 $m1|(m1 \ll 16)$ 扩展成 0x55555555，m2、m4 同理。有了掩码之后，主体用分治——先把 32 位分成小组，每组数自己的 1 个数，再不断合并相邻小组，依次变成 4 位、8 位、16 位一组，最后合成总数。每一层的关键是：用掩码把相邻两组的统计值分开、对齐，再相加。

第一步是每 2 位统计。一组 2 位只有两个位置，要把这两个位置的值取出来相加。用掩码 m1（每个 2 位里低位为 1、高位为 0）， $x \& m1$ 保留偶数位， $(x \gg 1) \& m1$ 把奇数位移到偶数位位置再保留，两者相加。这样每个 2 位字段里存的就是这 2 位里 1 的个数（0~2），而且不会溢出到下一组，因为最大是 2，2 位能存 0~3。第二步是每 4 位统计。掩码 m2（每个 4 位里低 2 位为 1），用 $(x \& m2) + ((x \gg 2) \& m2)$ 把每个 4 位里的低 2 位和高 2 位的统计值相加，得到 0~4，存在 4 位字段里。第三步是每 8 位统计。掩码 m4（每个 8 位里低 4 位为 1），用 $(x + (x \gg 4)) \& m4$ 合并高低 4 位，得到 0~8。这里先加再与，是因为低 4 位的和可能进位到高 4 位，加上掩码后只保留低 4 位的结果。第四步是合并 16 位和 32 位。依次 $x = x + (x \gg 8)$ 和 $x = x + (x \gg 16)$ ，把 16 位、32 位的统计合并，结果存在低 6 位里，最后用 $x \& 0x3F$ 取出。

19、题目要求：把 32 位整数的二进制位完全反转。

实现思路：

应该用分治法，从相邻小段开始，每一轮把位置逐层交换。先交换相邻 1 位，再交换相邻 2 位、4 位、8 位、16 位，每一步用掩码保留要移动的位，移位后相或合并。

首先通过 0xFF 拼接并逐层异或构造不同的掩码： $m8 = 0xFF | (0xFF \ll 16) = 0x00FF00FF$ ：每 2 个字节的低字节为 1。 $m16 = 0xFF | (0xFF \ll 8) = 0x0000FFFF$ ：前两个字节为 0、后两个字节为 1。 $m4 = m8 \wedge (m8 \ll 4) = 0x0F0F0F0F$ ：每个字节的低 4 位为 1。 $m2 = m4 \wedge (m4 \ll 2) = 0x33333333$ ：每 4 位的低 2 位为 1。 $m1 = m2 \wedge (m2 \ll 1) = 0x55555555$ ：奇数位为 1。

每一步的交换都是用掩码提取要交换的两组位，一组右移、一组左移，再合并。比如交换相邻 1 位时，用掩码 m1（每个 2 位里低位为 1）， $(x \gg 1) \& m1$ 把奇数位移到偶数位位置， $(x \& m1) \ll 1$ 把偶数位移到奇数位位置，相或完成交换。后续步骤同理，只是距离变成 2、4、8、16，最终实现完全反转。

代码验证截图：

```
tanjiayun@localhost:~/ICS2026/Tanjiayun-Lab-01$ make clean
rm -f *.o btest fshow ishow *~
tanjiayun@localhost:~/ICS2026/Tanjiayun-Lab-01$ make all
gcc -O -Wall -fwrapv -m32 -o btest bits.c btest.c decl.c tests.c -lm
gcc -O -Wall -fwrapv -m32 -o fshow fshow.c
gcc -O -Wall -fwrapv -m32 -o ishow ishow.c
```

```
tanjiayun@localhost:~/ICS2026/Tanjiayun-Lab-01$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 10/12 operators
logicalShift: 9/20 operators
swapNibblePairs: 11/18 operators
secondLowestZeroBit: 8/8 operators
oddParity: 12/56 operators
rotateRightBits: 15/16 operators
roundEvenPow2: 11/24 operators
midpointTowardFirst: 26/32 operators
isBetweenEitherOrder: 42/48 operators
mul5Sat: 7/30 operators
classifyAdd3: 33/52 operators
floatScaleThreeHalves: 31/60 operators
floatRoundEven: 26/65 operators
float_i2f: 19/40 operators
bitCount: 24/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

```
● tanjiayun@localhost:~/ICS2026/Tanjiayun-Lab-01$ ./btest
```

Score	Rating	Errors	Function
1	1	0	signMask
2	2	0	bitXor
3	3	0	negativePart
4	4	0	copyByteWithin
4	4	0	logicalShift
4	4	0	swapNibblePairs
4	4	0	secondLowestZeroBit
5	5	0	oddParity
5	5	0	rotateRightBits
5	5	0	roundEvenPow2
5	5	0	midpointTowardFirst
7	7	0	isBetweenEitherOrder
7	7	0	mul5Sat
7	7	0	classifyAdd3
7	7	0	floatScaleThreeHalves
10	10	0	floatRoundEven
10	10	0	float_i2f
10	10	0	bitCount
10	10	0	bitReverse
Total points: 110/110			